

# RBD 破壊 — 最初の本物のシミュレーション

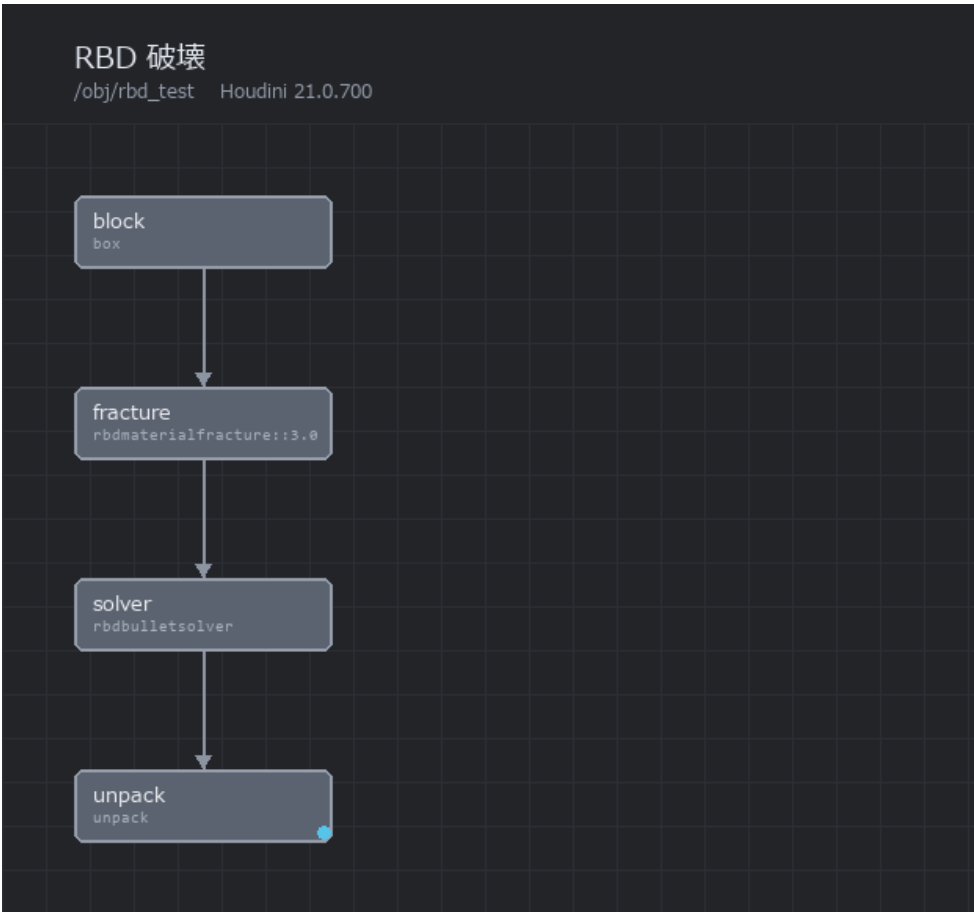
2026-09-13 | Houdini 21.0.700 | /obj/rbd\_test | ノード 4 / 接続 3

## 何を作ったか

実験013で、剛体（RBD）は SOP レベルのノードが47種あり、パーティクルより入口が易しいと分かった。ここまでの実験はすべて「1フレームの計算」だったが、今回は前のフレームの結果を次に渡していく本物のシミュレーション。

シミュレーションは結果が複雑で、目で見ても正しいかどうか判断しづらい。そこで検証の軸を1つ決めた。剛体は変形しないので、砕けても落ちてもぶつかっても、全部の破片の体積の合計は変わらないはず。これが崩れたら計算のどこかがおかしい。

## ノードの接続



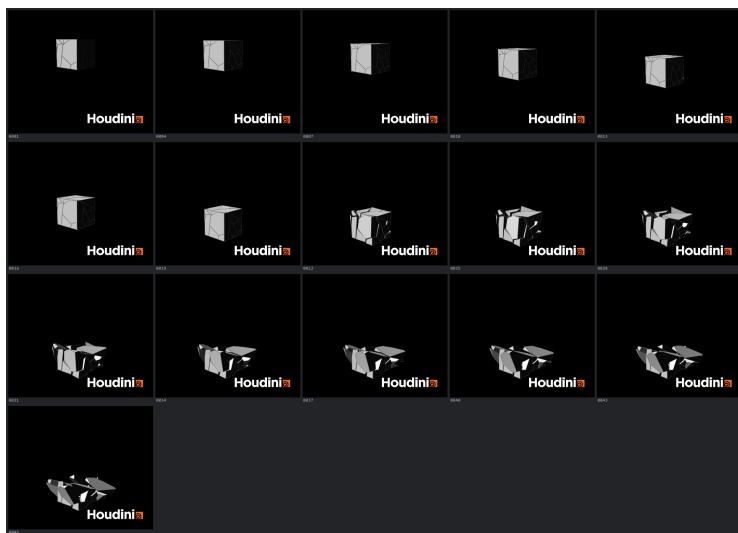
スクリプトが実際に構築したネットワーク。水色の点は表示フラグ。

| ノード      | タイプ                      | デフォルトから変更したパラメータ        |
|----------|--------------------------|-------------------------|
| block    | box                      | scale = 2.0<br>ty = 4.0 |
| fracture | rbdmaterialfracture::3.0 | (変更なし)                  |

| ノード    | タイプ             | デフォルトから変更したパラメータ                                                                                                                                                                                                                                                                                                                                                                 |
|--------|-----------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| solver | rbdbulletsolver | folder10 = 1<br>folder13 = 1<br>folder21 = 1<br>folder24 = 1<br>folder26 = 1<br>folder26_1 = 1<br>folder26_2 = 1<br>folder27 = 1<br>folder3 = 1<br>folder34 = 1<br>folder35 = 1<br>folder38 = 1<br>folder4 = 1<br>folder42 = 1<br>folder44 = 1<br>folder45 = 1<br>folder47 = 1<br>folder5 = 1<br>folder6 = 1<br>folder7 = 1<br>folder9 = 1<br>setupfolder_2 = 1<br>useground = 1 |
| unpack | unpack          | (変更なし)                                                                                                                                                                                                                                                                                                                                                                           |

## 高さ4から落として砕く（16コマ）

2×2×2 の箱を rbdmaterialfracture（コンクリート）で25個の破片に砕き、rbdbulletsolverで落とした。地面はソルバの機能で用意している。



16コマ。落下中は塊のまま、着地して初めて碎ける

## フレームごとの状態

「横の広がり」はバウンディングボックスの水平方向の大きさ。元の箱は 2.0 なので、これが増えていけば破片が散らばったということ。

| フレーム | 体積の合計   | 最下点    | 横の広がり  | 高さ     | 計算にかかった秒 |
|------|---------|--------|--------|--------|----------|
| 1    | 8.00000 | 3.0000 | 2.0000 | 2.0000 | 0.38     |

| フレーム | 体積の合計   | 最下点     | 横の広がり  | 高さ     | 計算にかかった秒 |
|------|---------|---------|--------|--------|----------|
| 4    | 8.00000 | 2.9208  | 2.0000 | 2.0000 | 0.02     |
| 10   | 8.00000 | 2.3028  | 2.0000 | 2.0000 | 0.03     |
| 20   | 8.00000 | -0.0492 | 2.0589 | 2.0331 | 0.05     |
| 30   | 8.00000 | -0.0225 | 3.7483 | 2.0181 | 0.03     |
| 46   | 8.00000 | -0.0219 | 5.6408 | 1.9498 | 0.05     |

## 検証: 体積は保たれているか

| 調べたこと           | 結果                  |
|-----------------|---------------------|
| 元の箱 (2×2×2) の体積 | 8.00000             |
| 砕いた直後の合計        | 8.00000 (差 0.00000) |
| 全フレームでの振れ幅      | 0.0000000 (0.0000%) |

## わかったこと

- ・体積は完全に保たれた。全6フレームで 8.00000 ちょうど、振れ幅は 0.0000000。元の箱の体積 8.0 とともに完全に一致する。砕いても落としてもぶつかっても体積が変わらないので、剛体シミュレーションとして正しく動いている。
- ・これは目で見ても分からない種類の検証。破片が散らばった画を見ても「正しく計算できているか」は判断できないが、体積が保たれていれば少なくとも物質は増減していないと言える。実験012で boolean に使った検算と同じ考え方。
- ・落下中は砕けない。フレーム1から10まで、横の広がり は 2.0000 のまま完全に変化しない。砕いてはあるが、破片はくっついたまま1つの塊として落ちる。
- ・着地の瞬間に広がり始める。フレーム20で最下点が -0.0492 になり（地面を少し突き抜ける）、同時に広がり が 2.0589 に増える。その後 30で 3.7483、46で 5.6408。元の 2.8倍まで散らばった。
- ・同時に全体の高さは 2.0000 から 1.9498 に下がる。積み上がっていたものが平たく崩れたということ。
- ・最下点は着地時に -0.0492 まで沈み、その後 -0.0219 で落ち着く。破片は地面を少し突き抜けたまま止まる。ソルバが衝突判定に持たせている余裕（マージン）によるもので、厳密に地面の上に乗るわけではない。
- ・rbdmaterialfracture のコンクリートモードで、2×2×2 の箱が 25個の破片になった（282点 / 191面）。破片の区別は name というプリミティブ属性で管理されている。
- ・計算は速かった。最初のフレームが 0.38秒、以降は 0.02～0.05秒。全部で1秒程度。シミュレーションといっても、この規模なら実験を回すのに支障はない。
- ・rdbbullet solver の既定は substeps = 10、cacheenabled = 1。1フレームを10回に分けて計算しており、結果は自動でキャッシュされる。2回目以降のフレーム送りが速いのはこのため。
- ・地面はソルバの useground で用意できる。別途ジオメトリを作る必要はない。

## 次にやること

- ・substeps を減らすと結果がどう変わるか。突き抜けや不安定さが出るはず

- ・破片の数を増やしたときの計算時間の伸び方を測る
- ・ Vellum（布）でも同じように、保存されるべき量を見つけて検証する